

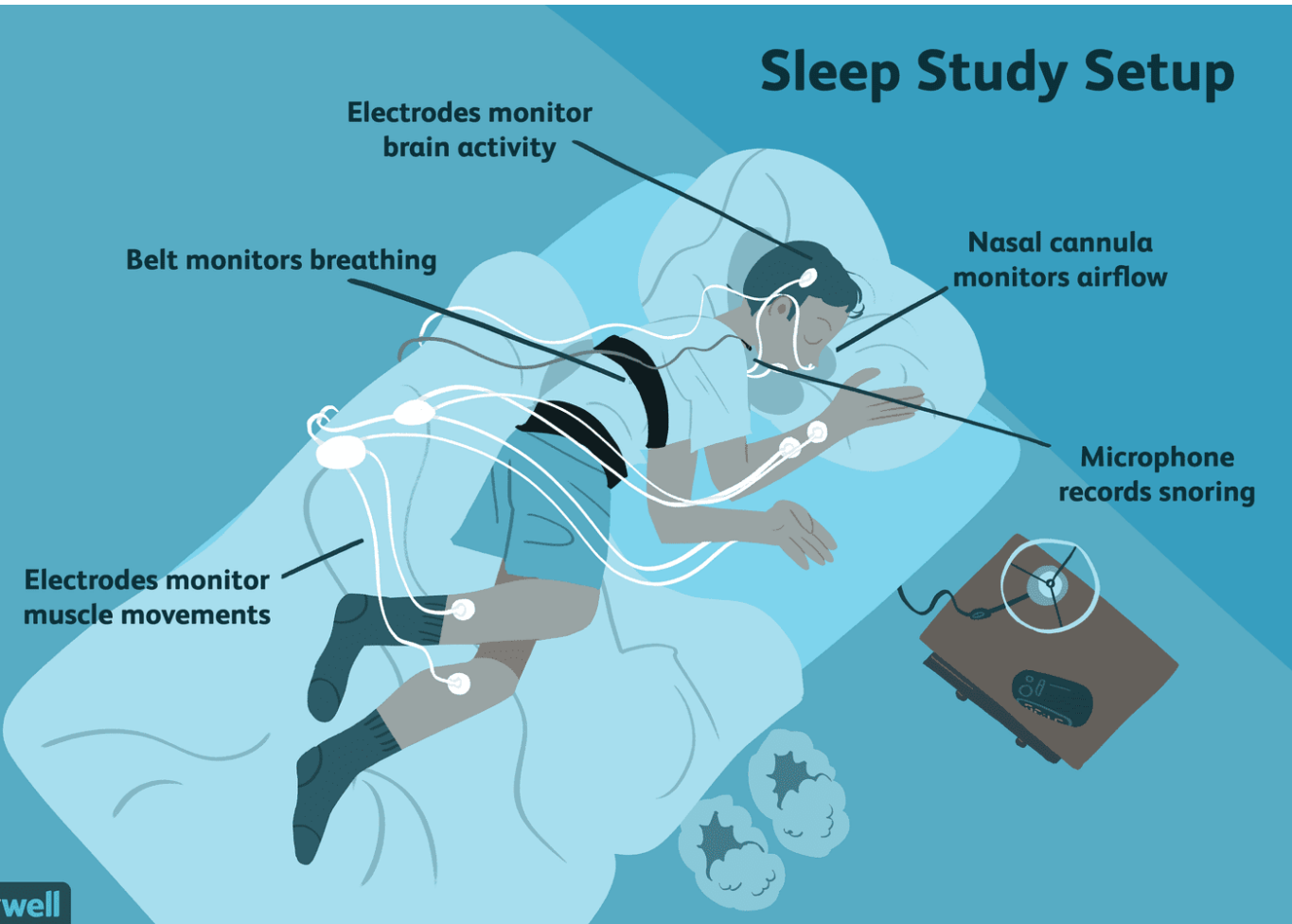
Class03h_Tutorial_PSG_34CSV_MLP

- **Tutorial Demo of Python code development**
- **Application : Sleep Stage Prediction**
- **The full sleep duration (several hours) is divided into many 30-second epochs. Within each epoch, we want to predict the sleep stage based on the Heart Rate (HR), HRV, Movement (Active_Score) and a time index.**
- **Inputs: HR, HRV, Active_Score, time**
- **Output: Light, Deep, REM, Wake**

Where does the data come from ?

Answer: PSG

The signals for sleep stage prediction



EEG (brain)

ECG (heart)

EOG (eye)

EMG leg/muscle

Breathing/SpO2

PPG (blood)

Finger movement
Time

PSG

(polysomnography)
Sleep analysis

HR

HRV

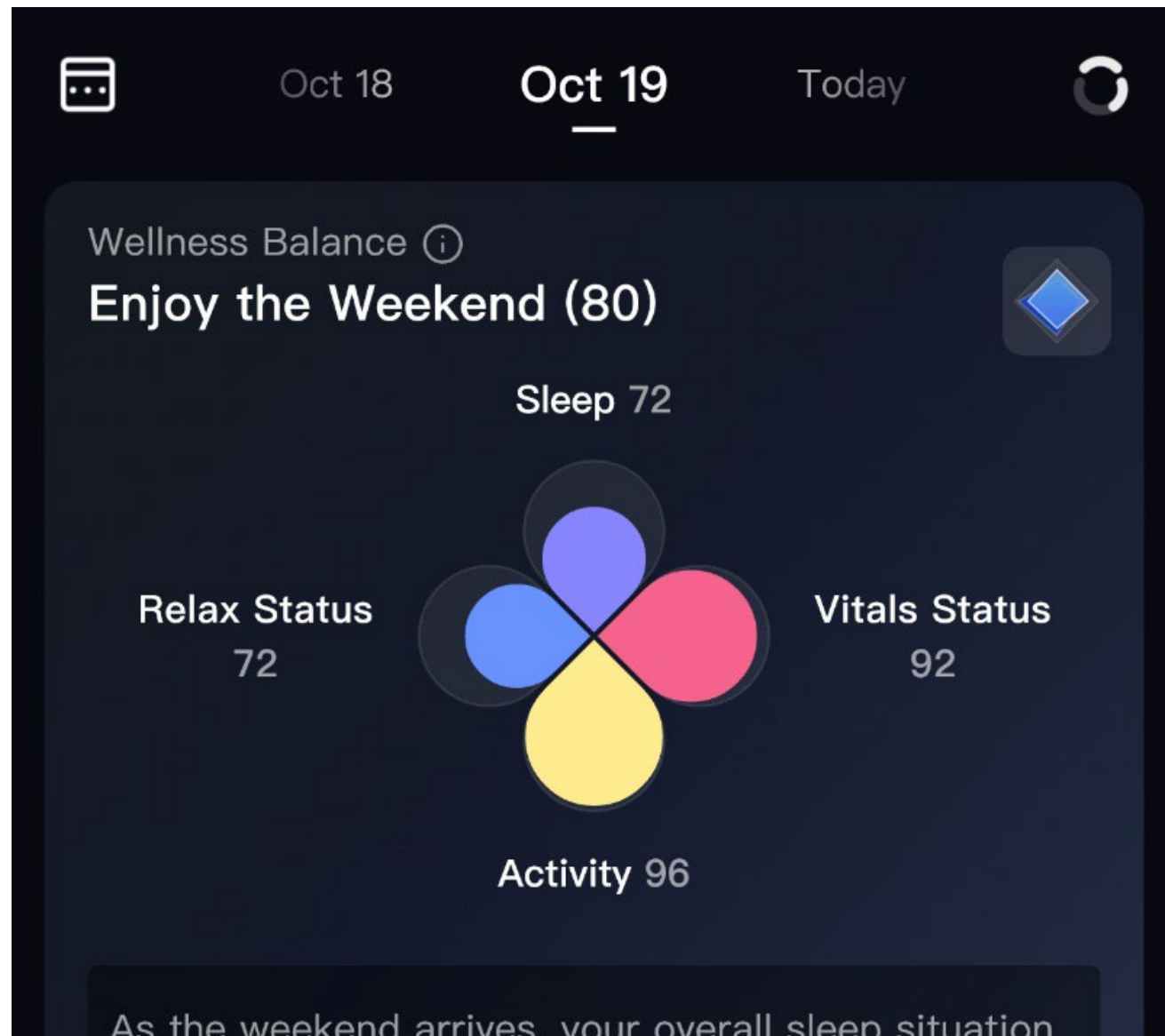
SpO2

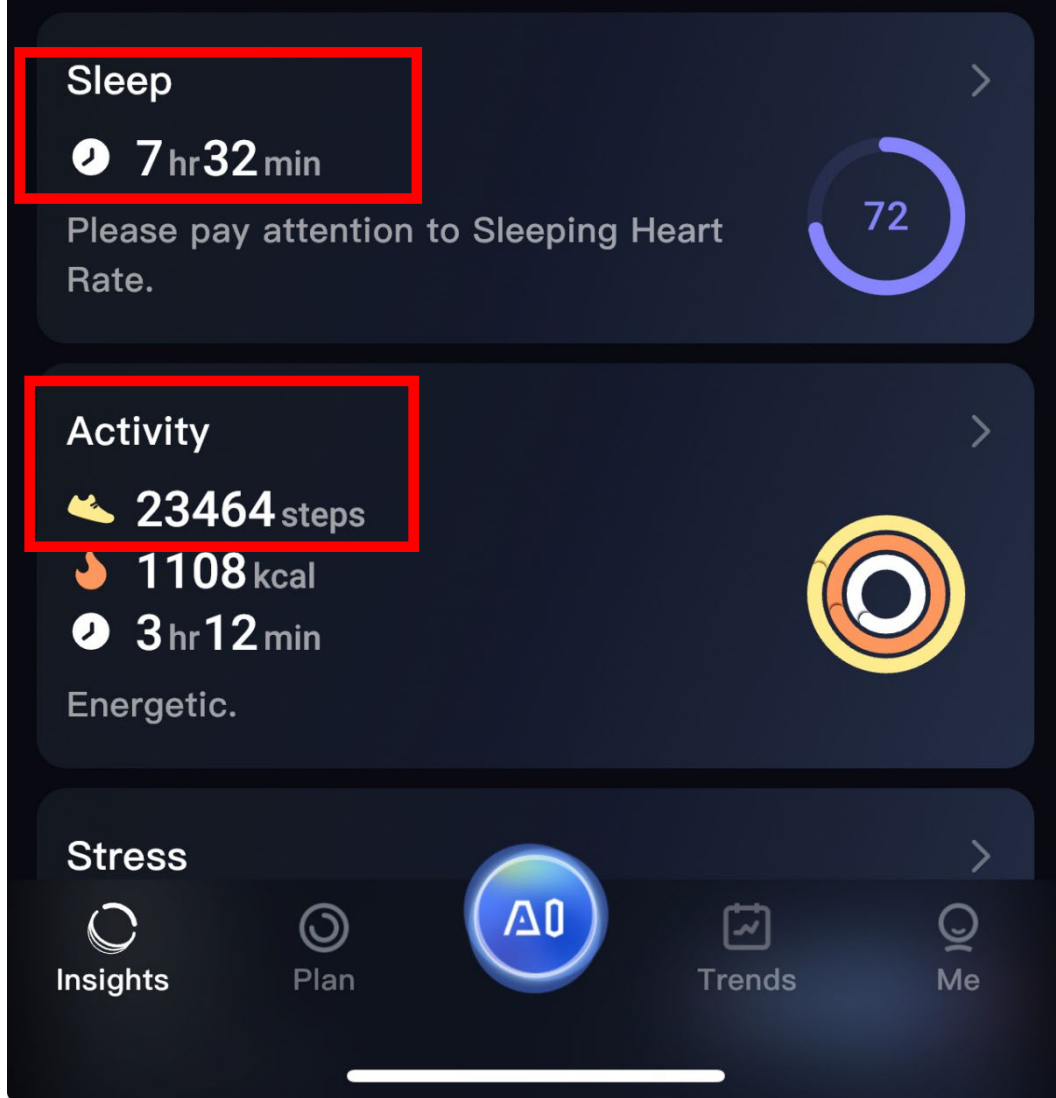
Active_Score
time

The state-of-the-art hardware

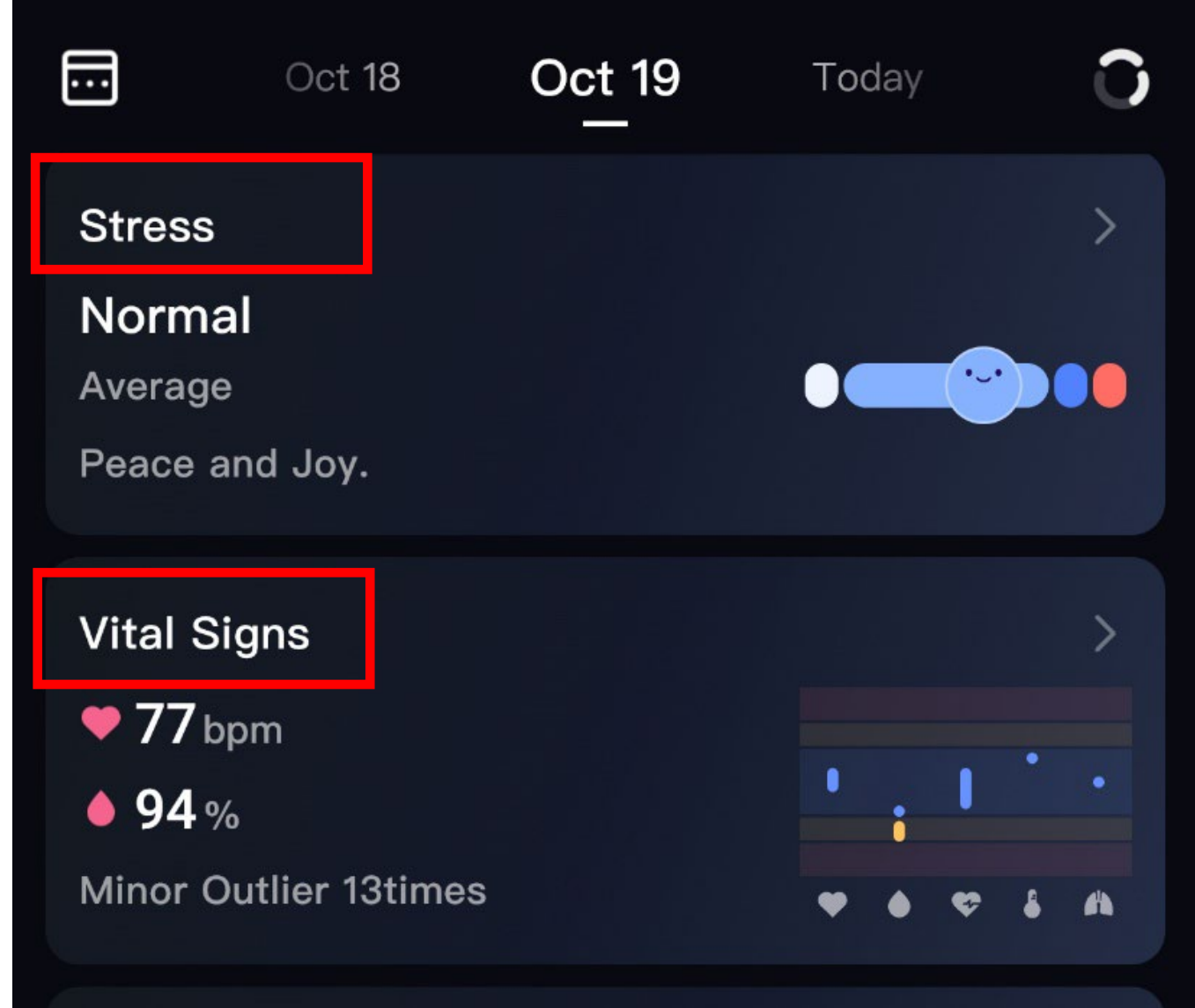


The state-of-the-art Apps





1. Sleep
2. Activity



3. Stress
4. Vital signs



Sleep ▾



Oct 18

Oct 19

Today



Sleep Score Factors >

Time Asleep ⓘ

Reference: 6–10 hr

7hr32min

✓ Normal

Time in Bed ⓘ

Reference: 6–10 hr

8hr21min

1:01 AM–9:21 AM ✓ Normal

Sleep Efficiency ⓘ

Reference: 80%–100%

90%

✓ Normal

Time Asleep

7 hr 32 min

Sleep Efficiency

90%

Sleep Stages ⓘ

Sleep HR 



1:01 AM

3:06 AM

5:11 AM

7:16 AM

9:21 AM



Awake



REM



Light Sleep



Deep Sleep

Movement ⓘ



Low Moderate High

Awake 7.0%

34min

REM 10.7%

52min

Light Sleep 63.8%

5hr10min

Deep Sleep 18.5%

1hr30min

Optimal Interval

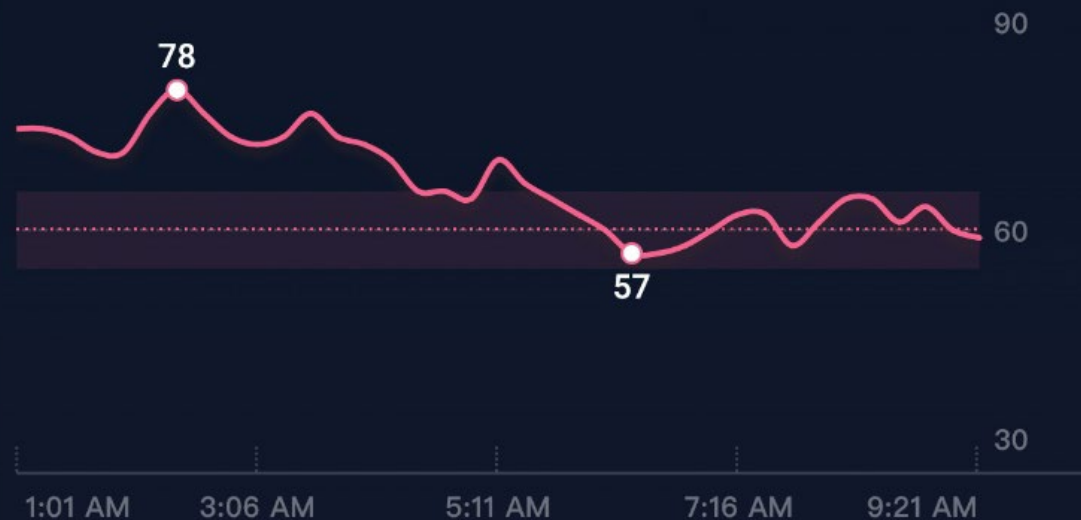
Heart Rate (bpm) ⓘ

Average

66

Recent 7-night Average

60



Recent 7-night Average Sleep Standard HR ⓘ

HRV (ms) ⓘ
HRV

Average

46

Heart Rate Variability

Recent 7-night Average

58



..... Recent 7-night Average

Tips Compared with your sleep over the past 7 days, your HRV has dropped significantly, usually indicating that your recovery has reduced due to physical or mental factors. If you experience any discomfort, please seek medical advice promptly.

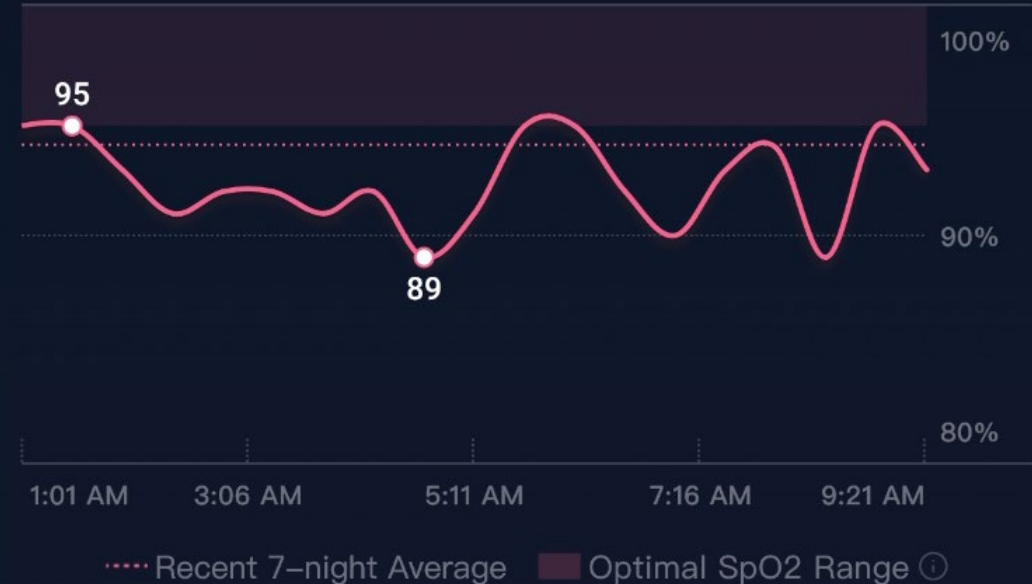
SpO2 (%) ⓘ

Average

92

Recent 7-night Average

94



..... Recent 7-night Average ■ Optimal SpO2 Range ⓘ

Tips During sleep, the overall blood oxygen trend is abnormal. Nasal stuffiness, sleep environment, and other underlying conditions (such as, allergy, etc.) may result in reduced blood oxygen. Please see a doctor promptly if you experience any discomfort.

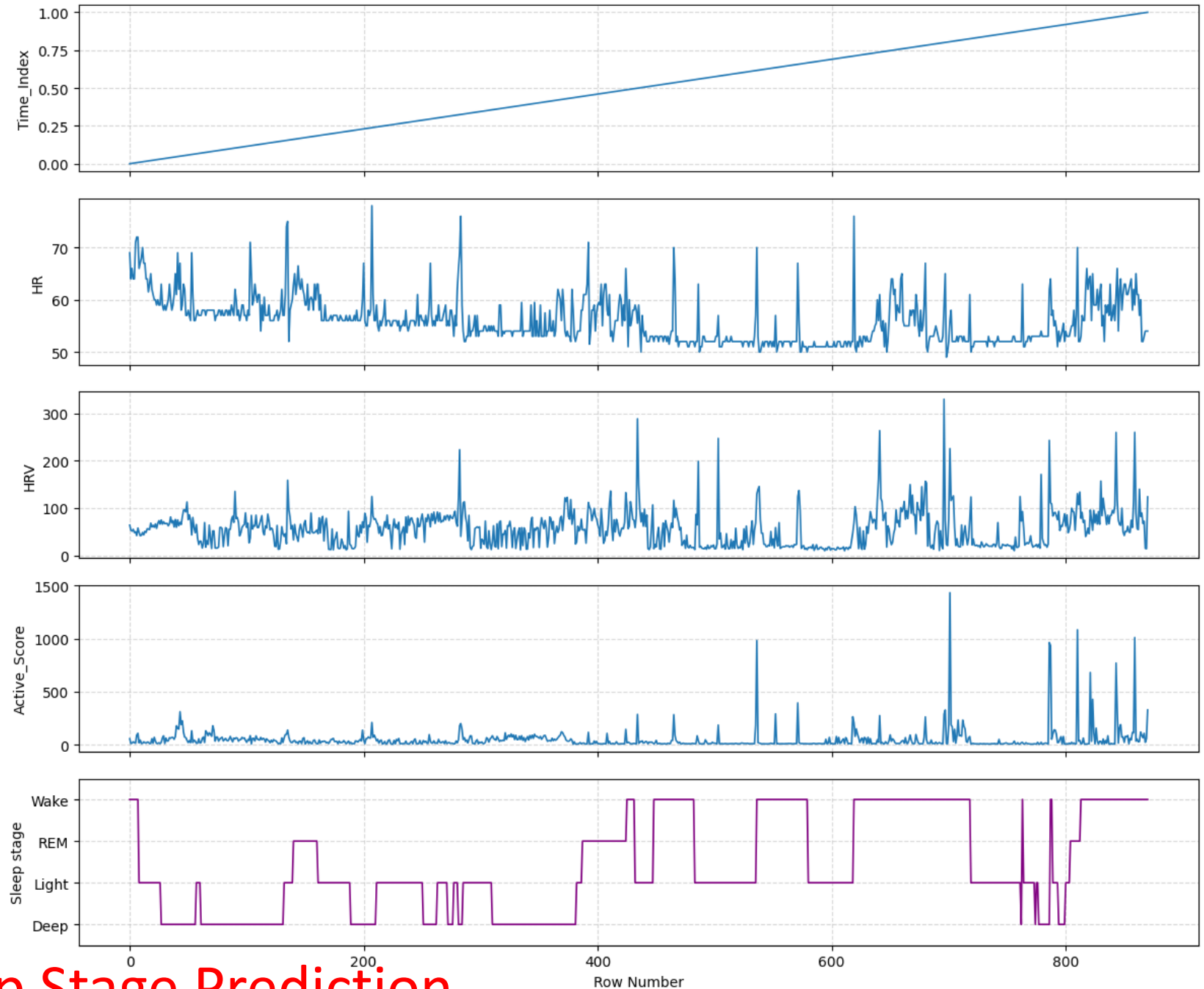
10001.csv

Display of one of 34 .csv files,
which is 100001.csv

Inputs are HR, HRV and
Active_Score/Movement and
Time_index.

Output is the sleep label.

Number of rows (each 30-sec
epoch): 871



Sleep Stage Prediction

Class03h_Tutorial_PSG_34CSV_MLP.ipynb

```
▶ # Evaluation of MLP model for Sleep Stage Prediction
# August 27, 2026
# Dr. G. Pang

# Background:
# 34 .csv sleep files are obtained from a PSG folder
# Each file contains many rows of HR, HRV, Movement, Time index
# with the labeled sleep stage of an 30s epoch.

# 30 files are used for training.
# 4 files for testing.
# HR, HRV, Movement and Time are used for sleep stage prediction.

# train_34CSV_4inputs.csv (from 30 files) contains 29413 rows for training.
# Each row contains HR + HRV + Movement + Time_index + labeled_sleep_stage.
# test_34CSV_4inputs.csv (from 4 files) contains 4632 rows for testing.
```

```
# Import the useful facilities
from sklearn.preprocessing import OneHotEncoder, StandardScaler, LabelEncoder
from sklearn.metrics import classification_report, confusion_matrix, accuracy_score, f1_score

import glob
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
```

```
# This is needed to read in the two .csv files for training and testing
# placed in your own folder.
from google.colab import drive
drive.mount('/content/drive')
```

Mounted at /content/drive

```
# Import the relevant libraries
```

```
import random
```

```
import numpy as np
```

```
import pandas as pd
```

```
import torch
```

```
import torch.nn as nn
```

```
from torch.utils.data import Dataset, DataLoader
```

```
from sklearn.metrics import f1_score, cohen_kappa_score, confusion_matrix
```

```
def set_seed(seed=42):
```

```
    random.seed(seed)
```

```
    np.random.seed(seed)
```

```
    torch.manual_seed(seed)
```

```
    torch.cuda.manual_seed_all(seed)
```

```

class SleepEpochDataset(Dataset):
    """
    CSV format per row:
    column 0 -> HR value
    column 1 -> HRV value
    column 2 -> movement/Active_score
    column 3 -> time index
    column 4 -> label (0,1,2,3)

    Returns one sample per row:
    x: (4,)
    y: scalar label
    """
    def __init__(self, csv_path, normalize=True, stats=None):
        df = pd.read_csv(csv_path, header=None)
        if df.shape[1] != 5:
            raise ValueError(f"Expected 5 columns, got {df.shape[1]}")

        data = df.values.astype(np.float32)

        self.x = data[:, :4]
        self.y = data[:, 4].astype(np.int64)

        if normalize:
            if stats is None:
                self.stats = {
                    "mean": self.x.mean(axis=0),
                    "std": self.x.std(axis=0) + 1e-6,
                }
            else:
                self.stats = stats

        self.x = (self.x - self.stats["mean"]) / self.stats["std"]
    else:
        self.stats = None

```

cont'd

```

def __len__(self):
    return len(self.y)

def __getitem__(self, idx):
    x = torch.tensor(self.x[idx], dtype=torch.float32)
    y = torch.tensor(self.y[idx], dtype=torch.long)
    return x, y

```

```
class SleepMLP(nn.Module):
    """
    Pure MLP baseline:
    Input: 4 features
    Hidden: 2 layers with ReLU + dropout
    Output: 4-class logits
    """
    def __init__(self, input_dim=4, hidden_dim=128, num_classes=4, dropout=0.3):
        super().__init__()
        self.net = nn.Sequential(
            nn.Linear(input_dim, hidden_dim),
            nn.ReLU(),
            nn.Dropout(dropout),

            nn.Linear(hidden_dim, hidden_dim),
            nn.ReLU(),
            nn.Dropout(dropout),

            nn.Linear(hidden_dim, num_classes),
        )

    def forward(self, x):
        # x: (B, 4)
        return self.net(x)
```

```
def compute_accuracy(logits, targets):  
    preds = torch.argmax(logits, dim=-1)  
    return (preds == targets).float().mean().item()  
  
def train_one_epoch(model, loader, optimizer, criterion, device):  
    model.train()  
    total_loss = 0.0  
    total_acc = 0.0  
  
    for x, y in loader:  
        x = x.to(device)  
        y = y.to(device)  
  
        optimizer.zero_grad()  
        logits = model(x)  
        loss = criterion(logits, y)  
        loss.backward()  
        optimizer.step()  
  
        total_loss += loss.item()  
        total_acc += compute_accuracy(logits, y)  
  
    return total_loss / len(loader), total_acc / len(loader)
```

```

@torch.no_grad()
def evaluate(model, loader, criterion, device, save_predictions_path=None):
    model.eval()
    total_loss = 0.0
    total_acc = 0.0

    all_true = []
    all_pred = []

    for x, y in loader:
        x = x.to(device)
        y = y.to(device)

        logits = model(x)
        loss = criterion(logits, y)

        preds = torch.argmax(logits, dim=-1)

        total_loss += loss.item()
        total_acc += compute_accuracy(logits, y)

        all_true.append(y.cpu().numpy())
        all_pred.append(preds.cpu().numpy())

    y_true = np.concatenate(all_true)
    y_pred = np.concatenate(all_pred)

    cm = confusion_matrix(y_true, y_pred, labels=[0, 1, 2, 3])
    macro_f1 = f1_score(y_true, y_pred, average="macro")
    kappa = cohen_kappa_score(y_true, y_pred)

```

cont'd

```

if save_predictions_path is not None:
    df_pred = pd.DataFrame({
        "y_true": y_true,
        "y_pred": y_pred
    })
    df_pred.to_csv(save_predictions_path, index=False)

metrics = {
    "loss": total_loss / len(loader),
    "acc": total_acc / len(loader),
    "macro_f1": macro_f1,
    "kappa": kappa,
    "cm": cm,
    "y_true": y_true,
    "y_pred": y_pred,
}
return metrics

```

```
def count_parameters(model):  
    return sum(p.numel() for p in model.parameters() if p.requires_grad)
```

Next is the main() function:

```
def main(seedIndex):  
    set_seed(seedIndex)  
    print("-----")  
    print("seedIndex = ", seedIndex)  
  
    train_csv = "/content/drive/My Drive/Colab Notebooks/data/train_34CSV_4inputs.csv"  
    test_csv = "/content/drive/My Drive/Colab Notebooks/data/test_34CSV_4inputs.csv"  
  
    batch_size = 512  
    lr = 1e-3  
    weight_decay = 1e-4  
    epochs = 50  
  
    save_test_predictions = True  
    prediction_csv_path = "test_epoch_predictions_mlp.csv"  
  
    device = torch.device("cuda" if torch.cuda.is_available() else "cpu")  
    print("Using device:", device)
```

cont'd

```
train_dataset = SleepEpochDataset(  
    csv_path=train_csv,  
    normalize=True,  
    stats=None,  
)
```

```
test_dataset = SleepEpochDataset(  
    csv_path=test_csv,  
    normalize=True,  
    stats=train_dataset.stats,  
)
```

```
train_loader = DataLoader(  
    train_dataset,  
    batch_size=batch_size,  
    shuffle=True,  
    num_workers=0,  
    pin_memory=True,  
)
```

```
test_loader = DataLoader(  
    test_dataset,  
    batch_size=batch_size,  
    shuffle=False,  
    num_workers=0,  
    pin_memory=True,  
)
```

```
model = SleepMLP(  
    input_dim=4,  
    hidden_dim=128,  
    num_classes=4,  
    dropout=0.3,  
)
```

```
print("-----")  
print("Trainable parameters of SleepMLP :", count_parameters(model))  
print("-----")
```

```
criterion = nn.CrossEntropyLoss()  
optimizer = torch.optim.AdamW(model.parameters(), lr=lr, weight_decay=weight_decay)
```

```
best_macro_f1 = -1.0  
best_results = None
```

```
# The main loop for training
for epoch in range(1, epochs + 1):
    train_loss, train_acc = train_one_epoch(
        model, train_loader, optimizer, criterion, device
    )

    eval_results = evaluate(
        model,
        test_loader,
        criterion,
        device,
        save_predictions_path=prediction_csv_path if save_test_predictions else None
    )

    print(
        f"Epoch {epoch:03d} | "
        f"Train Loss: {train_loss:.4f} | Train Acc: {train_acc:.4f} | "
        f"Test Loss: {eval_results['loss']:.4f} | "
        f"Test Acc: {eval_results['acc']:.4f} | "
        f"Macro-F1: {eval_results['macro_f1']:.4f} | "
        f"Kappa: {eval_results['kappa']:.4f}"
    )
```

```

if eval_results["macro_f1"] > best_macro_f1:
    best_macro_f1 = eval_results["macro_f1"]
    best_results = eval_results

torch.save(
    {
        "model_state_dict": model.state_dict(),
        "train_stats": train_dataset.stats,
    },
    "best_mlp_sleep_staging.pt",
)

if save_test_predictions:
    pd.DataFrame({
        "y_true": best_results["y_true"],
        "y_pred": best_results["y_pred"]
    }).to_csv(prediction_csv_path, index=False)

# End of the main loop and display of result

```

```

print("\nBest test results (selected by Macro-F1):")
print(f"Accuracy      : {best_results['acc']:.4f}")
print(f"Macro-F1       : {best_results['macro_f1']:.4f}")
print(f"Cohen kappa    : {best_results['kappa']:.4f}")
print("Confusion matrix:")
print(best_results["cm"])
print("\nRows = true class, Columns = predicted class")
print("Class order: 0, 1, 2, 3")

if save_test_predictions:
    print(f"\nSaved per-epoch predictions to: {prediction_csv_path}")

```



```
# To execute MLP 5 times, using seed 42,43,44,45,46
for i in range(42,47):
    main(i)
```

```
*** -----
seedIndex = 42
Using device: cpu
-----
Trainable parameters of SleepMLP : 17668
-----
/usr/local/lib/python3.13/dist-packages/torch/utils/data/dataloader.py:77
  super().__init__(loader)
Epoch 001 | Train Loss: 1.2295 | Train Acc: 0.4714 | Test Loss: 1.1747 |
/usr/local/lib/python3.13/dist-packages/torch/utils/data/dataloader.py:77
  super().__init__(loader)
Epoch 002 | Train Loss: 1.1894 | Train Acc: 0.4933 | Test Loss: 1.1723 |
/usr/local/lib/python3.13/dist-packages/torch/utils/data/dataloader.py:77
  super().__init__(loader)
Epoch 003 | Train Loss: 1.1784 | Train Acc: 0.4957 | Test Loss: 1.1604 |
/usr/local/lib/python3.13/dist-packages/torch/utils/data/dataloader.py:77
```

```
Epoch 050 | Train Loss: 1.0956 | Train Acc: 0.5295 | Test Loss: 1.2079 | Test Acc: 0.4570
```

Best test results (selected by Macro-F1):

Accuracy : 0.4676

Macro-F1 : 0.3549

Cohen kappa : 0.1541

Confusion matrix:

```
[[ 57  391  149   82]
 [ 88 1374  607  248]
 [ 20  385  228   93]
 [ 92  312   96 410]]
```

Rows = true class, Columns = predicted class

Class order: 0, 1, 2, 3

```
seedIndex = 43
```

```
Using device: cpu
```

```
-----
Trainable parameters of SleepMLP : 17668
-----
```

```
/usr/local/lib/python3.13/dist-packages/torch/utils/data/dataloader.py:775: UserWarning: The dataloader is instantiated with a custom seed. This is not recommended as it may lead to non-reproducible results.
  super().__init__(loader)
```

```
Epoch 001 | Train Loss: 1.2249 | Train Acc: 0.4841 | Test Loss: 1.1710 | Test Acc: 0.4570
```

```
/usr/local/lib/python3.13/dist-packages/torch/utils/data/dataloader.py:775: UserWarning: The dataloader is instantiated with a custom seed. This is not recommended as it may lead to non-reproducible results.
  super().__init__(loader)
```

```
Epoch 002 | Train Loss: 1.1861 | Train Acc: 0.4960 | Test Loss: 1.1602 | Test Acc: 0.4570
```



```
# Result reporting (after 5 different trials using 5 seeds)
# For calculation of mean and std
import numpy as np

data = [0.4676, 0.4904, 0.4503, 0.4928, 0.5037]

# NumPy library
mean_np = np.mean(data)
std_np = np.std(data, ddof=0) # Set ddof=0 for population (not sample)

print(f"Mean: {mean_np:.3f}")
print(f"Standard Deviation: {std_np:.3f}")

*** Mean: 0.481
    Standard Deviation: 0.019
```



```
# To examine the model and count the number of parameters in details
model = SleepMLP()
print(model)
print("-----")
print("Trainable parameters of SleepMLP:", count_parameters(model))
print("-----")
def print_param_breakdown(model):
    total = 0
    for name, param in model.named_parameters():
        p = param.numel()
        print(f"{name:40s} : {p:,}")
        total += p
    print("-" * 50)
    print(f"Total: {total:,}")

print_param_breakdown(model)
```

```

... SleepMLP(
    (net): Sequential(
      (0): Linear(in_features=4, out_features=128, bias=True)
      (1): ReLU()
      (2): Dropout(p=0.3, inplace=False)
      (3): Linear(in_features=128, out_features=128, bias=True)
      (4): ReLU()
      (5): Dropout(p=0.3, inplace=False)
      (6): Linear(in_features=128, out_features=4, bias=True)
    )
  )

```

```

-----
Trainable parameters of SleepMLP: 17668
-----

```

net.0.weight	: 512
net.0.bias	: 128
net.3.weight	: 16,384
net.3.bias	: 128
net.6.weight	: 512
net.6.bias	: 4

```

-----
Total: 17,668

```

The data files are train_34CSV_4inputs.csv and test_34CSV_4inputs.csv
(29413, 5) for training ; (4632, 5) for testing

Run 5 times, seed = 42 to 46

Accuracy Mean: 0.481 and Standard Deviation: 0.019

Model structure: 4 → 128 → 128 → 4

Summary of learning items in the tutorial demo :

- Colab connecting to Google Drive
- class SleepEpochDataset(Dataset) to read in data
- class SleepMLP gives the machine learning model structure
- many auxiliary functions: train, evaluate, metrics, count_parameters
- main() has the main training loop with use of seed, display of results
- calculation of mean/std of final result
- printout the number of parameters layer by layer
- Avoid train-test data contamination (the info from the training set is completely separated from the test set. Hence, do not use the “split”).

The End of Tutorial Demo